

Instance Segmentation

Instance Segmentation

- 1、 [Model Introduction](#)
- 2、 [Instance Segmentation: Image](#)
 - [Result Preview](#)
- 3、 [Instance Segmentation: Video](#)
 - [Result Preview](#)
- 4、 [Instance Segmentation: Real-time Detection](#)
 - 4.1、 [USB Camera](#)
 - [Result Preview](#)
- [Reference](#)

This section demonstrates **Ultralytics** Instance Segmentation on images, videos, and real-time detection using Python.

1、 Model Introduction

Instance segmentation goes a step further than object detection. It involves identifying individual objects in an image and separating them from the rest of the image.

The output of an instance segmentation model is a set of masks or contours outlining each object in the image, along with a class label and confidence score for each object. Instance segmentation is very useful when you need to know not only where objects are located in an image, but also their exact shapes.

2、 Instance Segmentation: Image

Use yolo26n-seg_ncnn_model to predict images in the ultralytics26 folder.

Enter the code folder:

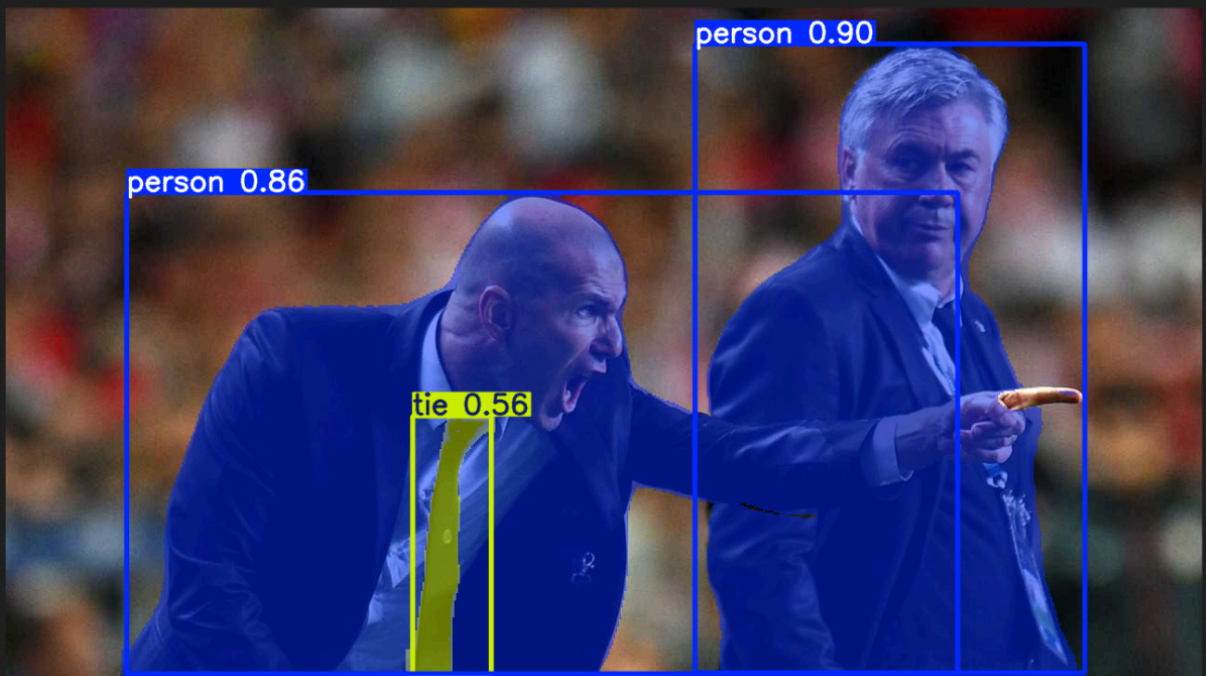
```
cd /home/sunrise/ultralytics26/yahboom_demo
```

Run the code:

```
python3 02.segmentation_image.py
```

Result Preview

Output image location for yolo detection: /home/sunrise/ultralytics26/output/



Example code:

```
from ultralytics import YOLO

# Load a model
model = YOLO("/home/sunrise/ultralytics26/yolo26n-seg_ncnn_model")
# model = YOLO("/home/sunrise/ultralytics26/yolo26n-seg.pt")

# Run batched inference on a list of images
results = model("/home/sunrise/ultralytics26/assets/zidane.jpg") # return a list of Results
objects

# Process results list
for result in results:
    # boxes = result.boxes # Boxes object for bounding box outputs
    masks = result.masks # Masks object for segmentation masks outputs
    # keypoints = result.keypoints # Keypoints object for pose outputs
    # probs = result.probs # Probs object for classification outputs
    # obb = result.obb # Oriented boxes object for OBB outputs
    result.show() # display to screen
    result.save(filename="/home/sunrise/ultralytics26/output/zidane_output.jpg") # save to
disk
```

3、 Instance Segmentation: Video

Use yolo26n-seg_ncnn_model to predict videos in the ultralytics26 folder (not Ultralytics built-in videos).

Enter the code folder:

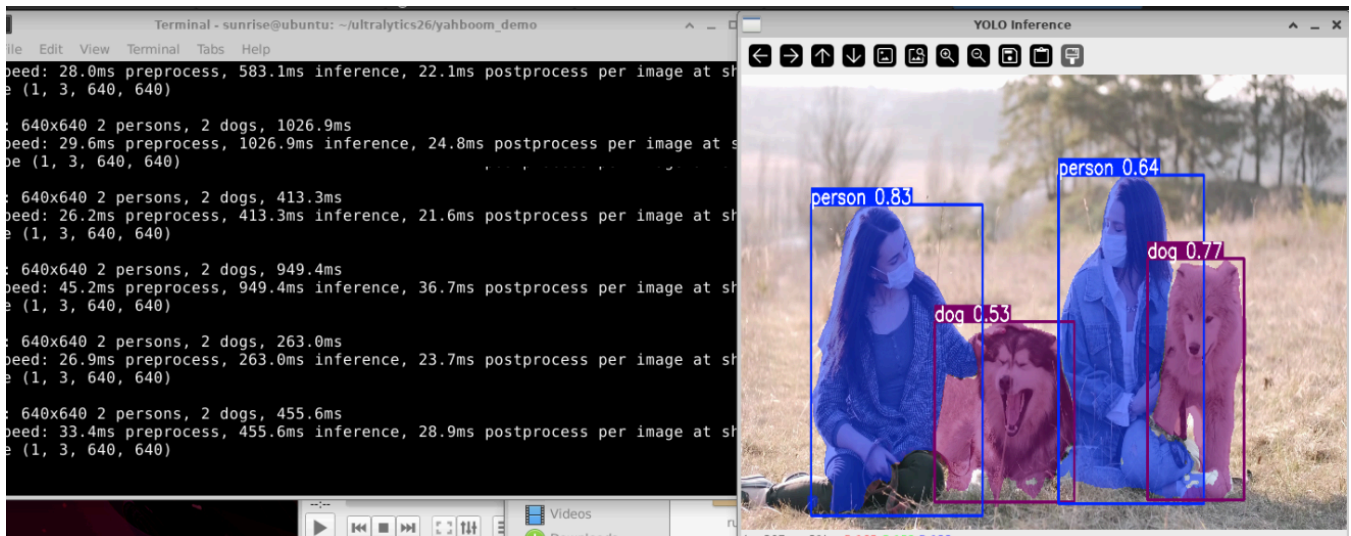
```
cd /home/sunrise/ultralytics26/yahboom_demo
```

Run the code:

```
python3 02.segmentation_video.py
```

Result Preview

Output video location for yolo detection: /home/sunrise/ultralytics/ultralytics/output/



Example code:

```
import cv2
from ultralytics import YOLO

# Load the YOLO model
model = YOLO("/home/sunrise/ultralytics26/yolo26n-seg_ncnn_model")
# model = YOLO("/home/sunrise/ultralytics26/yolo26n-seg.pt")

# Open the video file
video_path = "/home/sunrise/ultralytics26/videos/people_animals.mp4"
cap = cv2.VideoCapture(video_path)

# Get the video frame size and frame rate
frame_width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
frame_height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
fps = int(cap.get(cv2.CAP_PROP_FPS))

# Define the codec and create a VideoWriter object to output the processed video
output_path = "/home/sunrise/ultralytics26/output/02.people_animals_output.mp4"
fourcc = cv2.VideoWriter_fourcc(*'mp4v') # You can use 'XVID' or 'mp4v' depending on your platform
out = cv2.VideoWriter(output_path, fourcc, fps, (frame_width, frame_height))

# Loop through the video frames
while cap.isOpened():
    # Read a frame from the video
```

```

success, frame = cap.read()

if success:
    # Run YOLO inference on the frame
    results = model(frame)

    # Visualize the results on the frame
    annotated_frame = results[0].plot()

    # Write the annotated frame to the output video file
    out.write(annotated_frame)

    # Display the annotated frame
    cv2.imshow("YOLO Inference", cv2.resize(annotated_frame, (640, 480)))

    # Break the loop if 'q' is pressed
    if cv2.waitKey(1) & 0xFF == ord("q"):
        break
else:
    # Break the loop if the end of the video is reached
    break

# Release the video capture and writer objects, and close the display window
cap.release()
out.release()
cv2.destroyAllWindows()

```

4、 Instance Segmentation: Real-time Detection

4.1、 USB Camera

Use yolo26n-seg_ncnn_model to predict USB camera feed.

Enter the code folder:

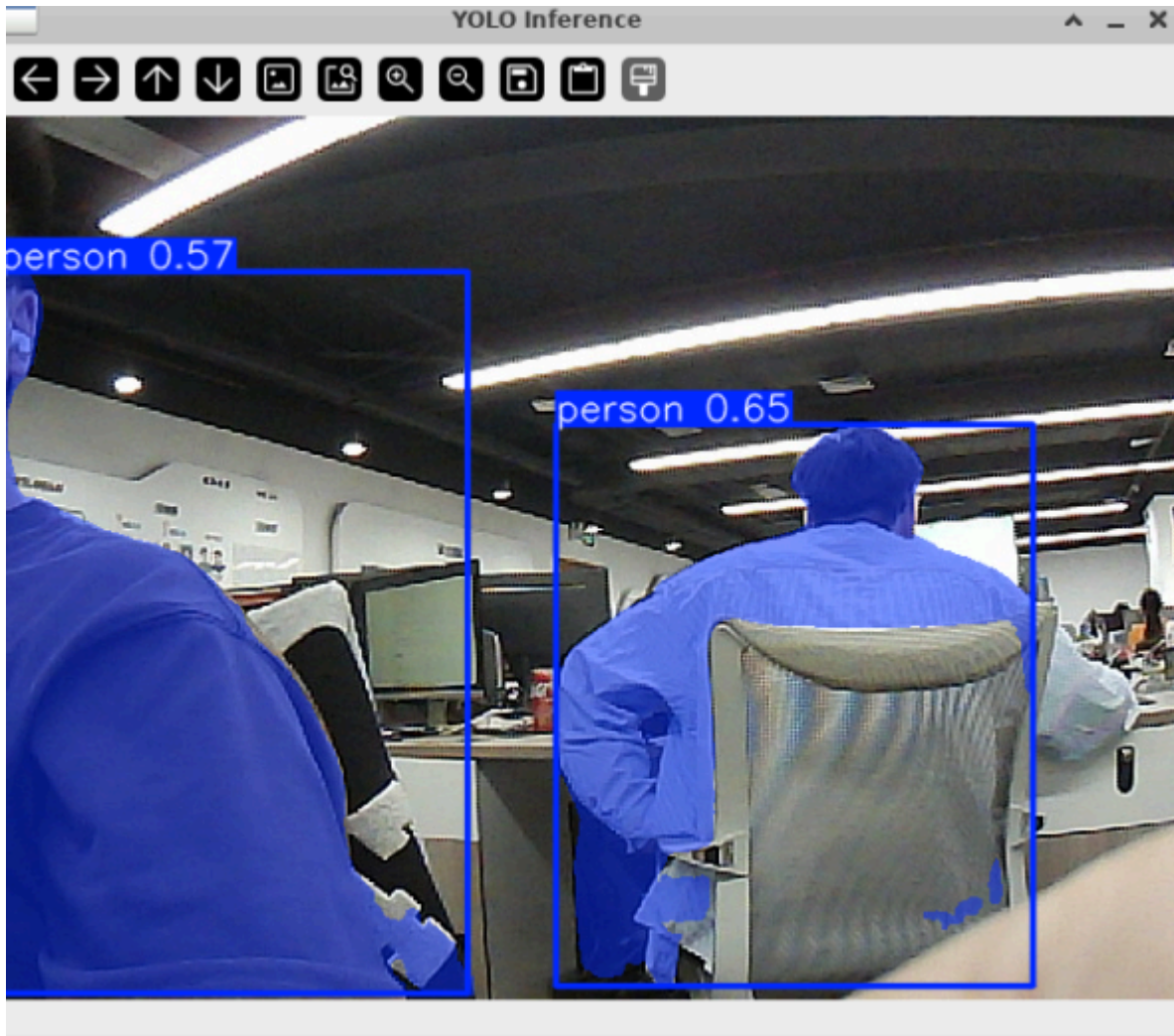
```
cd /home/sunrise/ultralytics26/yahboom_demo
```

Run the code: Click the preview window and press 'q' to terminate the program!

```
python3 02.segmentation_camera_usb.py
```

Result Preview

Output video location for yolo detection: /home/sunrise/ultralytics26/output/



Example code:

```
import cv2
from ultralytics import YOLO

# Load the YOLO model
model = YOLO("/home/sunrise/ultralytics26/yolo26n-seg_ncnn_model")
# model = YOLO("/home/sunrise/ultralytics26/yolo26n-seg.pt")

# Open the camera
cap = cv2.VideoCapture(0)
cap.set(6, cv2.VideoWriter_fourcc('M', 'J', 'P', 'G'))
cap.set(cv2.CAP_PROP_FRAME_WIDTH, 640)
cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 480)

# Get the video frame size and frame rate
frame_width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
frame_height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
fps = int(cap.get(cv2.CAP_PROP_FPS))

# Define the codec and create a Videowriter object to output the processed video
output_path = "/home/sunrise/ultralytics26/output/02.segmentation_camera_usb.mp4"
```

```

fourcc = cv2.VideoWriter_fourcc(*'mp4v') # You can use 'XVID' or 'mp4v' depending on your
platform
out = cv2.VideoWriter(output_path, fourcc, fps, (frame_width, frame_height))

# Loop through the video frames
while cap.isOpened():
    # Read a frame from the video
    success, frame = cap.read()

    if success:
        # Run YOLO inference on the frame
        results = model(frame)

        # Visualize the results on the frame
        annotated_frame = results[0].plot()

        # Write the annotated frame to the output video file
        out.write(annotated_frame)

        # Display the annotated frame
        cv2.imshow("YOLO Inference", cv2.resize(annotated_frame, (640, 480)))

        # Break the loop if 'q' is pressed
        if cv2.waitKey(1) & 0xFF == ord("q"):
            break
    else:
        # Break the loop if the end of the video is reached
        break

# Release the video capture and writer objects, and close the display window
cap.release()
out.release()
cv2.destroyAllWindows()

```

Reference

[Instance Segmentation - Ultralytics YOLO Documentation](#)